

Capitolul 11

Elemente fundamentale ale limbajului Prolog

11.1. Scurt istoric al limbajului Prolog

Limbajul Prolog (PROgrammation en LOGique) a fost elaborat la Universitatea din Marsilia în jurul anului 1970, ca instrument pentru programarea și rezolvarea problemelor ce implicau reprezentări simbolice de obiecte și relații dintre acestea.

Primul interpretor de Prolog a fost scris în limbajul Fortran. Prolog a fost inclus de Japonia ca limbaj de bază în planul de dezvoltare a sistemelor de generația a cincea.

Ca și Lisp, Prolog este un limbaj cu fundament matematic. Dar, spre deosebire de Lisp, specializat în definirea de funcții, Prolog permite definirea și prelucrarea relațiilor.

Limbajul Prolog are un pronunțat caracter descriptiv: un program Prolog este o colecție de definiții ce descriu relații sau funcții de calculat. Execuția programului constă din utilizarea definițiilor pentru a găsi o relație sau un obiect corespunzător unor specificații date. Soluția problemelor nu se mai vede ca o execuție pas cu pas a unei secvențe de instrucțiuni.

Prolog are un câmp de aplicații foarte larg: baze de date relaționale, inteligență artificială, logică matematică, demonstrarea de teoreme, sisteme expert, rezolvarea de probleme abstracte sau ecuații simbolice, etc.

În această parte vom studia implementarea Turbo Prolog 2.0 a firmei Borland International.

11.2. De la limbajul natural la programele Prolog

În Prolog, soluțiile se obțin prin deducții logice plecând de la lucruri deja cunoscute. Un program Prolog nu este o înșiruire de acțiuni de îndeplinit, ci o colecție de fapte împreună cu regulile după care se pot trage concluzii din aceste fapte. Prolog se bazează pe un subset al logicii predicatelor. Oferă o sintaxă foarte apropiată limbajului natural. O facilitate importantă a limbajului Prolog este că, în plus față de găsirea logică a răspunsurilor la întrebări, poate trata alternativele și să găsească toate soluțiile, nu numai una dintre ele.

Iată câteva exemple de afirmații din limbajul natural și ‘traducerile’ respective în logica predicatelor:

- O mașină rapidă este distractivă. distractiv(mașină_rapidă).
- O mașină mare este utilă. util(mașină_mare).

- Lui Bill îi place o mașină dacă mașina este distractivă. place(bill, Mașină) if distractiv(Mașină).

11.2.1. Propoziții: fapte și reguli

Un programator Prolog va defini obiecte și relații, apoi va defini reguli despre când aceste relații vor fi adevărate.

11.2.1.1. Fapte: ceea ce se cunoaște

În Prolog, o relație între obiecte se numește fapt. În limbajul natural o relație este simbolizată de o propoziție. În Prolog, o relație este sumarizată printr-o frază simplă ce constă din numele relației urmat între paranteze de obiectele participante la relație. Ca și în limbajul natural, un fapt se termină cu punct. Exemplul următor prezintă mai multe fapte exprimând noțiunea de plăcere din limbajul natural:

- Vasile o place pe Ana.
- Ana îl place pe Vasile.
- Lui Vasile îi plac câinii.

Iată aceste fapte scrise în Prolog:

- place(vasile, ana).
- place(ana, vasile).
- place(vasile, câini).

Faptele pot exprima și proprietăți. De exemplu, afirmația “Renault este o mașină” se scrie în Prolog astfel:

- mașină(renault).

11.2.1.2. Reguli: ce anume se poate deduce din faptele date

Regulile permit deducerea de fapte din alte fapte. Altfel spus, o regulă este o concluzie care se știe că este adevărată atunci când alte concluzii sau fapte sunt adevărate. Iată câteva reguli privind relație de plăcere de mai sus:

- Anei îi place tot ceea ce îi place lui Vasile.
- Mariei îi place orice mașină.

Fiind date aceste reguli, se poate deduce din faptele cunoscute deja, că:

- Anei îi place de Ana.
- Mariei îi place Renault-ul.

Iată cum se scriu aceste reguli în Prolog:

- place(ana, Ceva) if place(vasile, Ceva).
- place(maria, Ceva) if mașină(Ceva).

11.2.2. Întrebări

Odată ca am scris o serie de fapte, putem începe să punem întrebări relativ la aceste fapte. În limbajul natural am întreba:

- Ii place lui Vasile de Ana?

În sintaxă Prolog, vom scrie

- place(vasile, ana).
La această întrebare, Prolog va răspunde
- Yes

Întrebarea

- Ce îi place lui Vasile?

va fi redactată în Prolog astfel:

- place(vasile, Ce).

Notați că sintaxa Prolog nu se schimbă atunci când puneți o întrebare. Este important de notat că al doilea obiect (*Ce*) începe cu **majusculă**, în timp ce primul (*vasile*) începe cu **minusculă**. Aceasta deoarece *vasile* este o constantă (o valoare cunoscută) iar *Ce* este o variabilă. În general, constantele încep cu literă mică, iar variabilele cu literă mare.

La întrebarea de mai sus Prolog va raspunde

- Ce = câini
Ce = ana
2 Solutions

Ca un alt exemplu, la întrebarea

- place(ana, Ce).

Prolog va răspunde

- Ce = caini
Ce = ana
Ce = vasile
3 Solutions

11.2.3. Asamblarea faptelor, regulilor și întrebărilor

Să analizăm următorul program Prolog:

```
place(ellen, tenis).  
place(john, fotbal).  
place(tom, basket).  
place(eric, inot).  
place(mark, tenis).  
place(bill, Activitate) :-  
    place(tom, Activitate).
```

La întrebarea

?place(bill, basket).

Prolog va răspunde cu

true

11.2.4. Variabile: propoziții generale

În Prolog, variabilele ne ajută să scriem fapte generale și reguli, și să cerem răspuns la întrebări. Necesitatea variabilelor apare mai ales în cazul întrebărilor compuse. Afirmații generale tipice ar putea fi:

- Lui Bill îi plac aceleași lucruri ca și lui Kim.
- Temperatura curentă este caldă.

Iată aceste lucruri scrise în Prolog:

- place(bill, Lucruri) and place(kim, Lucruri).
- temperatura(TempCrt) and cald(TempCrt).

11.3. Elemente ale limbajului Prolog

11.3.1. Clauze (fapte și reguli)

Principal, în Prolog există doar două tipuri de fraze: fapte și reguli. Aceste fraze sunt cunoscute sub numele de clauze.

11.3.1.1. Fapte

Un fapt reprezintă o **singură apariție** fie a unei proprietăți a unui obiect, fie a unei relații între obiecte. Un fapt este de sine stătător; Prolog nu are nevoie să caute altceva pentru confirmarea unui fapt, și faptul poate fi folosit ca bază pentru inferențe.

11.3.1.2. Reguli

Constructorul din Prolog care descrie ceea **ce se poate deduce din alte informații este regula**. O regulă este o proprietate sau o relație care se știe că este adevărată atunci când alte relații sunt adevărate.

De exemplu:

1. diana_poate_mânca(Mâncare_de_pe_meniu) if
vegetal(Mâncare_de_pe_meniu) and
pe_lista_doctorului(Mâncare_de_pe_meniu).

2. părinte(Persoana1, Persoana2) if
tata(Persoana1, Persoana2).
părinte(Persoana1, Persoana2) if
mama(Persoana1, Persoana2).

Se observă că ‘sau’ din limbajul natural se traduce în Prolog prin două reguli separate:

- Persoana1 este părintele lui Persoana2 dacă Persoana1 este tatăl lui Persoana2.
 - Persoana1 este părintele lui Persoana2 dacă Persoana1 este mama lui Persoana2.
3. poate_cumpăra(Nume, Model) if
persoană(Nume) and
mașină(Model) and
place(Nume, Model) and
de_vânzare(Model).

11.3.2. Predicate (relații)

Numele simbolic al unei relații se numește numele predicatului. Obiectele la care se referă se numesc argumentele sale.

Iată câteva exemple de predicate Prolog cu zero sau mai multe argumente:

- pred(integer, symbol)
- person(last, first, gender)
- run
- insert_mode
- birthday(firstName, lastName, date)

11.3.3. Variabile (clauze generale)

Într-o întrebare simplă, puteți utiliza variabile pentru a cere sistemului Prolog să găsească informații. De exemplu, întrebarea

place(X, tenis).

folosește litera X ca variabilă, pentru a indica o persoană necunoscută. Numele de variabile trebuie să înceapă cu literă mare, după care pot urma litere, cifre și caracterul underline (_).

11.3.4. Cum își primesc valori variabilele?

Ați observat că Prolog nu are instrucțiuni de atribuire. Variabilele în Prolog își primesc valorile prin potrivire cu constante din fapte sau reguli.

Până când o variabilă primește o valoare, ea este **liberă** (free); când variabila primește o valoare, ea este **legată** (bound). Dar ea stă legată atâta timp cât este necesar pentru a obține o soluție a problemei. Apoi, Prolog o dezleagă, face backtracking și caută soluții alternative.

Observație. Este important de reținut că nu se pot stoca informații prin atribuire de valori unor variabile. Variabilele sunt folosite ca parte a unui proces de potrivire, nu ca un tip de stocare de informații.

De exemplu, fie programul

```
place(ellen, citit).
place(john, calculatoare).
place(john, badminton).
place(leonard, badminton).
place(eric, înot).
place(eric, citit).
```

Fie întrebarea

```
place(Persoana, citit), place(Persoana, înot).
```

Prolog va rezolva cele două părți ale întrebării separat, căutând în clauzele programului, de sus în jos. În prima parte,

```
place(Persoana, citit)
```

variabila Persoana este liberă. Prolog caută un fapt care să se potrivească cu această primă parte:

```
place(ellen, citit)
```

Deci, Prolog leagă variabila liberă Persoana de constanta ellen. Acum, Prolog verifică a doua parte a întrebării. Deoarece Persoana este legată de ellen, trebuie verificat că

```
place(ellen, inot)
```

Prolog caută o potrivire a acestui fapt, dar nu găsește. Deci, a doua parte a întrebării nu este adevărată când Persoana este ellen.

Acum Prolog dezleagă Persoana, și încearcă să găsească o altă soluție a primei părți a întrebării, cu Persoana ca variabilă liberă. Mergând în continuare cu verificările, Prolog va lega Persoana de constanta eric. De această dată și partea a doua va fi satisfăcută, deci Prolog va returna

```
Persoana=eric
1 Solution
```

Observație. Mecanismul prin care Prolog încearcă să ‘potrivească’ partea din întrebare pe care dorește să o rezolve cu un anumit predicat se numește unificare. Ca rezultat al unificării, Prolog a legat variabila Persoana într-o primă fază de constanta ellen, și mai apoi de constanta eric.

Observație. Această întrebare este o întrebare compusă. O întrebare compusă poate să conțină și operatorul ‘or’. În acest caz, ‘and’ este mai prioritar decât ‘or’, iar părțile de verificat vor fi identificate în acord cu acest lucru.

11.3.5. Variabile anonime

Dacă prin punerea unei întrebări doriți să obțineți doar informații parțiale, puteți să utilizați variabilele anonime pentru a neglija acele părți care nu vă interesează. În Prolog, variabila anonimă este reprezentată de caracterul underline (_).

De exemplu, dacă dorim să știm dacă îi place cuiva să înoate, dar nu vrem să știm cui anume, vom întreba

`place(_, inot).`

De asemenea, variabilele anonime pot fi folosite în reprezentarea faptelor:

`are(_, pantofi).`

reprezintă

Toata lumea are pantofi.

11.3.6. Ce este o potrivire?

Iată câteva reguli ce vor explica termenul 'potrivire':

1. Structuri identice se potrivesc una cu alta
 - `parinte(joe, tammy)` se potrivește cu `parinte(joe, tammy)`
2. De obicei o potrivire implică variabile libere. Dacă X e liberă,
 - `parinte(joe, X)` se potrivește cu `parinte(joe, tammy)`
 - X este legat la tammy.
3. Dacă X este legat, se comportă ca o constantă. Cu X legat la tammy,
 - `parinte(joe, X)` se potrivește cu `parinte(joe, tammy)`
 - `parinte(joe, X)` NU se potrivește cu `parinte(joe, millie)`
4. Două variabile libere se potrivesc una cu alta.
 - `parinte(joe, X)` se potrivește cu `parinte(joe, Y)`

11.3.7. Modele de flux

În Prolog, legările de variabile se fac în două moduri: la intrarea în clauză sau la ieșirea din clauză. Direcția în care se leagă o valoare se numește ‘model de flux’. Când o variabilă este dată la intrarea într-o clauză, aceasta este un parametru de intrare (i), iar când o variabilă este

dată la ieșirea dintr-o clauză, aceasta este un parametru de ieșire (o). O anumită clauză poate să aibă mai multe modele de flux. De exemplu clauza

factorial (n,f)

(care va calcula factorialul lui n) poate avea următoarele modele de flux:

- (i,i) - verifică dacă $n! = f$;
- (i,o) - atribuie $f := n!$;
- (o,i) - găsește acel n pentru care $n! = f$.

Observație. Proprietatea unui predicat de a funcționa cu mai multe modele de flux depinde de abilitatea programatorului de a programa predicatul în mod corespunzător.

Observație. În programele Prolog, cuvântul 'if' se poate schimba cu ':-' (două-puncte liniuță), cuvântul 'and' se poate schimba cu ',' (virgulă) iar cuvântul 'or' se poate schimba cu ';' (punct-virgulă).

De exemplu, următoarele două porțiuni de cod sunt echivalente:

e_mai_bătrân (P1, P2) if	e_mai_bătrân(P1, P2) :-
vârsta(P1, V1) and	vârsta(P1, V1),
vârsta(P2, V2) and	vârsta(P2, V2),
V1 > V2.	V1 > V2.

11.3.8. Comentarii

Comentariile se includ între secvențele '/*' și '*/'. Alternativ, tot ceea ce apare pe o linie după caracterul '%' este comentariu:

```
/* comentariu */
```

```
% comentariu
```

```
/******  
/* comentariu */  
*****
```